CHAPTER 6

The Anonymous Lifetime `'_`

Chapter 3 showed that the compiler writes most lifetimes for you: elision fills in the lifetimes of references in a signature, by a fixed set of rules, without looking at the body. That works well as long as every lifetime is the lifetime of an `&`. It works less well for the many types that carry a lifetime parameter of their own, such as `SplitWhitespace<'a>`, `fmt::Formatter<'a>` or your own `Parser<'src>`, and it does not reach the hidden lifetime bound of a trait object at all.

The anonymous lifetime `'_` is how you say "there is a lifetime here, and I do not need to name it". It looks like a wildcard, and that impression is the source of most confusion about it: `'_` does not mean "any lifetime" and it does not mean "figure it out from the body". It means "do what elision would do", and what elision does depends on where you are. This chapter goes through its meaning in each position, its three everyday jobs, and the things it cannot do.

In this chapter you will learn:

- what `'_` means in each position where it is allowed, and why two occurrences of `'_` are never the same lifetime;
- how to make the hidden lifetime parameters of types such as `SplitWhitespace<'a>` and `Formatter<'a>` visible, and what the lints `mismatched_lifetime_syntaxes` and `elided_lifetimes_in_paths` report;
- when `'_` in an `impl` header is enough, and when the `impl` needs a name;
- how `+ '_` lets a boxed trait object borrow, in a return type, in a parameter and in an `impl dyn Trait` block;
- what `'_` means inside a function body;
- what `'_` cannot do, and a checklist for choosing between `'a`, `'_` and writing nothing.

6.1 `'_` is not a name

A named lifetime such as `'a` is a parameter: it is declared once (`fn longest<'a>`) and every occurrence refers to the same lifetime. That is the whole point of a name. `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` uses `'a` three times to say that the three references share one lifetime.

`'_` is not declared anywhere and cannot be referred to. Each occurrence is a placeholder that the compiler fills in, by the same rules it uses when you leave a lifetime out. In a signature, writing `&'_ str`

is exactly the same as writing `&str`; the difference is only that `'_` can also go where there is no `&` to leave the lifetime off, such as the lifetime argument of a type (`SplitWhitespace<'_>`) or the bound of a trait object (`dyn Fn() + '_`).

Meaning by position

What the compiler puts in place of `'_` depends on where it appears:

Position	`'_` means	Example
Parameter type	a fresh lifetime parameter	`it: SplitWhitespace<'_>`
Return type	the lifetime elision picks; E0106 if none	`-> SplitWhitespace<'_>`
`impl` header	a fresh, unnameable parameter of the `impl`	`impl Display for Words<'_>`
`dyn` bound, return type	the lifetime elision picks	`-> Box<dyn Fn() + '_>`
`dyn` bound, parameter	a fresh lifetime parameter	`Vec<Box<dyn FnMut() + '_>>`
`use<'_>` in a return type	the lifetime elision picks	`impl Sized + use<'_>`
A type in a function body	a lifetime to infer	`MutexGuard<'_, T>`

"A fresh lifetime parameter" is rule 1 of elision (Chapter 3): every elided lifetime in the parameters becomes a distinct lifetime parameter of the function, chosen by the caller. "The lifetime elision picks" is rules 2 and 3: if exactly one parameter carries a lifetime, or if there is a `&self` or `&mut self`, that lifetime is used for every elided lifetime in the output. A `'_` in the output takes part in those rules exactly like the missing lifetime of an `&`. When the rules have no answer, `'_` has none either, and the error is E0106, as we will see.

The table also has holes. `'_` is not allowed in struct fields, in bounds and `where` clauses, in the associated types of an `impl`, or inside an `impl Trait` in argument position. The section *What `'_` cannot do* below shows the errors.

Two `'_` are never the same lifetime

Because every `'_` in the parameters is a fresh lifetime, writing it twice does not tie two positions together. Here is a function that remembers words in a list (`code/compile_fail/ch06_never_equal.rs`):

```
pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
    seen.push(word);
}
```

The compiler rejects it:

```
error: lifetime may not live long enough
 --> compile_fail/ch06_never_equal.rs:7:5
  |
6 | pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
  |                               -              - let's call the lifetime of this
reference `'1`
  |                               |
  |                               let's call the lifetime of this reference `'2`
7 |     seen.push(word);
  |     ^^^^^^^^^^^^^^^ argument requires that `'1` must outlive `'2`
  |
  = note: requirement occurs because of a mutable reference to `Vec<&str>`
```

```
  = note: mutable references are invariant over their type parameter
  = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
help: consider introducing a named lifetime parameter
  |
6 - pub fn remember(seen: &mut Vec<&'_ str>, word: &'_ str) {
6 + pub fn remember<'a>(seen: &mut Vec<&'a str>, word: &'a str) {
  |
```

The error has no code; its message is "lifetime may not live long enough". The compiler names the two anonymous lifetimes for us: `'2` is the lifetime of the `&str` items inside the vector, and `'1` is the lifetime of `word`. With explicit names, the signature is `fn remember<'v, 'w>(seen: &mut Vec<&'v str>, word: &'w str)`: two unrelated lifetimes, each chosen by the caller. Pushing `word` into the vector stores a `&'w str` where a `&'v str` is expected, which requires `'w` to outlive `'v` ("argument requires that `'1` must outlive `'2`"), and nothing in the signature promises that. The two notes explain why the compiler cannot simply shrink the vector's lifetime to fit: `seen` is a mutable reference, and `&mut T` is invariant in `T` (Chapter 4).

The help is the fix. To say "these two are the same", you need a name, used twice: `fn remember<'a>(seen: &mut Vec<&'a str>, word: &'a str)`. Read `'_` as "a lifetime exists here, and nothing else in the signature refers to it".

6.2 Job 1: making hidden lifetime parameters visible

Many types in the standard library carry a lifetime parameter, because they borrow something: `SplitWhitespace<'a>` and `Lines<'a>` borrow the string they split, `slice::Iter<'a, T>` borrows the slice, `MutexGuard<'a, T>` borrows the mutex, and `fmt::Formatter<'a>` borrows the output buffer it writes to. Rust lets you write these types in a signature without their lifetime argument, as `SplitWhitespace` instead of `SplitWhitespace<'_>`. The compiler then elides the lifetime as it would for an `&`. The lifetime is still there; it is only *hidden*.

The trouble is the reader. `fn words(text: &str) -> SplitWhitespace` looks as if it returned an owned value, but it returns a borrow of `text`, and the caller finds out only when the borrow checker objects to dropping `text` too early. The fix is to write the lifetime, and since nothing else needs to refer to it, `'_` is the way to write it.

The `mismatched_lifetime_syntaxes` lint

Since Rust 1.89, the compiler warns about the most misleading case by default. The program `code/compile_fail/ch06_hidden_return.rs` turns the warnings into errors with `#![deny(mismatched_lifetime_syntaxes)]`, so that they can be shown here. Its first function is the one above:

```
pub fn words(text: &str) -> SplitWhitespace {
    text.split_whitespace()
}
```

```
error: hiding a lifetime that's elided elsewhere is confusing
  --> compile_fail/ch06_hidden_return.rs:10:20
```

```
   |
10 | pub fn words(text: &str) -> SplitWhitespace {
   |                    ^^^^     ^^^^^^^^^^^^^^^^ the same lifetime is hidden here
   |                    |
   |                    the lifetime is elided here
   |
   = help: the same lifetime is referred to in inconsistent ways, making the signature
confusing
note: the lint level is defined here
  --> compile_fail/ch06_hidden_return.rs:5:9
   |
 5 | #![deny(mismatched_lifetime_syntaxes)]
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use `'_` for type paths
   |
10 | pub fn words(text: &str) -> SplitWhitespace<'_> {
   |                                             ++++
```

The lint sorts the ways of writing a lifetime into three kinds: *named* (`&'a str`, `Iter<'a, T>`), *elided* (`&str`, or `'_` anywhere), and *hidden* (a path such as `SplitWhitespace` whose lifetime argument is missing). It fires when one lifetime, connecting an input to the output, is written in two different kinds. Here the labels show both ends of the connection: "the lifetime is elided here" under `&str`, and "the same lifetime is hidden here" under `SplitWhitespace`. The help adds the missing `<'_>`. Without the `deny`, this is a warning and the program compiles; the contract is fine, only its spelling is "confusing".

The second function of the file mixes the named and the elided kinds:

```rust
pub fn trimmed<'a>(text: &'a str) -> &str {
    text.trim()
}
```

```
error: eliding a lifetime that's named elsewhere is confusing
  --> compile_fail/ch06_hidden_return.rs:16:38
   |
16 | pub fn trimmed<'a>(text: &'a str) -> &str {
   |                           --         ^^^^ the same lifetime is elided here
   |                           |
   |                           the lifetime is named here
   |
   = help: the same lifetime is referred to in inconsistent ways, making the signature
confusing
help: consistently use `'a`
   |
16 | pub fn trimmed<'a>(text: &'a str) -> &'a str {
   |                                       ++
```

By rule 2 of elision, the output is the lifetime of `text`, which the signature calls `'a`; leaving it out of the return type hides that fact one line away from where it was named. The help writes `&'a str`. The corrected signatures are in `code/examples/ch06_hidden_lifetimes.rs`:

```rust
// In the return type, `'_` is the lifetime elision picks: the one of `text`.
fn words(text: &str) -> SplitWhitespace<'_> {
    text.split_whitespace()
}

// In a parameter, `'_` is a fresh lifetime parameter, as if it were elided.
fn count_long(words: SplitWhitespace<'_>, min_len: usize) -> usize {
    words.filter(|w| w.len() >= min_len).count()
}
```

In `words`, `'_` sits in the return type and means the lifetime of `text`, by rule 2. In `count_long`, it sits in a parameter and means a fresh lifetime parameter, so `count_long` accepts a `SplitWhitespace` borrowing any string at all. Since `&'_ str` and `&str` are both of the elided kind, mixing them is consistent, and the lint stays silent:

```
// `&'_ str` and `&str` are both "elided": using one in the parameter and the
// other in the return type is consistent, and the lint stays silent.
fn trimmed(text: &'_ str) -> &str {
    text.trim()
}
```

> **NOTE**
>
> `mismatched_lifetime_syntaxes` was added in Rust 1.89 (August 2025), as a warn-by-default lint that replaced the earlier `elided_named_lifetimes`. Code that compiled without warnings on older compilers may warn now. The field guide this book grew out of showed `fn words_hidden(s: &str) -> SplitWhitespace` as a warning you had to opt into with `elided_lifetimes_in_paths`; since 1.89 it warns by default. The warning never changes what a program means.

The `elided_lifetimes_in_paths` lint

`mismatched_lifetime_syntaxes` only compares lifetimes that connect inputs with outputs. A hidden lifetime in a parameter, with no output to connect to, passes it. The most common case is the one every `Display` implementation contains, the `Formatter`. A second lint, `elided_lifetimes_in_paths`, catches every hidden lifetime parameter. It is *allow-by-default* (silent unless you ask for it) and belongs to the lint group `rust_2018_idioms`. The program `code/compile_fail/ch06_hidden_param.rs` denies it, and it reports two errors:

```
impl fmt::Display for Port {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "port {}", self.0)
    }
}

pub fn record(log: &Mutex<Vec<String>>, line: &str) -> usize {
    let mut guard: MutexGuard<Vec<String>> = log.lock().unwrap();
    guard.push(line.to_string());
    guard.len()
}
```

```
error: hidden lifetime parameters in types are deprecated
  --> compile_fail/ch06_hidden_param.rs:16:32
   |
16 |     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
   |                           -----^^^^^^^^^
   |                           |
   |                           expected lifetime parameter
   |
note: the lint level is defined here
  --> compile_fail/ch06_hidden_param.rs:7:9
   |
 7 | #![deny(elided_lifetimes_in_paths)]
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^
help: indicate the anonymous lifetime
   |
```

```
16 |     fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
   |                                          ++++

error: hidden lifetime parameters in types are deprecated
  --> compile_fail/ch06_hidden_param.rs:24:30
   |
24 |     let mut guard: MutexGuard<Vec<String>> = log.lock().unwrap();
   |                    ----------^------------ expected lifetime parameter
   |
help: indicate the anonymous lifetime
   |
24 |     let mut guard: MutexGuard<'_, Vec<String>> = log.lock().unwrap();
   |                               +++
```

The message is "hidden lifetime parameters in types are deprecated", and the label "expected lifetime parameter" points between `Formatter` and its missing `<'_>`. The help, "indicate the anonymous lifetime", writes it. The second error shows that the lint also looks inside function bodies: the guard's type in the `let` hides the lifetime of its borrow of `log`.

To enable the lint for a whole crate, add `#![warn(elided_lifetimes_in_paths)]` (or `#![warn(rust_2018_idioms)]`) at the top of `lib.rs` or `main.rs`, or put it in `Cargo.toml`:

```
[lints.rust]
elided_lifetimes_in_paths = "warn"
```

The example `code/examples/ch06_hidden_lifetimes.rs` denies the lint, and it compiles because every lifetime is visible. Its `Display` implementation is the one you should write out of habit:

```
struct Port(u16);

impl fmt::Display for Port {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "port {}", self.0)
    }
}
```

Running the example prints:

```
3 long words
2 entries: ["listen on port 8080 for incoming requests", "port 8080"]
```

Writing `<'_>` costs four characters and tells every reader of the signature that a borrow is involved. In a crate's public API, it is worth enabling the lint.

6.3 Job 2: `impl` headers

An `impl` block for a type with a lifetime parameter must declare that parameter, as `impl<'src> Parser<'src>` declares `'src` and then uses it. When nothing inside the `impl` mentions the lifetime, the declaration is noise, and `'_` replaces it: in an `impl` header, `'_` declares a fresh lifetime parameter of the `impl` that has no name. `impl fmt::Display for Words<'_>` means exactly `impl<'a> fmt::Display for Words<'a>`. The example `code/examples/ch06_impl_headers.rs` uses a `Words` type that hands out the words of a borrowed text:

```
/// The words of a borrowed text.
struct Words<'src> {
    rest: &'src str,
}
```

The `Display` implementation and the helper method `is_done` never mention the text's lifetime, so `'_` is enough:

```
// Nothing here mentions the lifetime: `'_` is enough.
impl fmt::Display for Words<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Words({:?})", self.rest.trim())
    }
}

impl Words<'_> {
    fn is_done(&self) -> bool {
        self.rest.trim().is_empty()
    }
}
```

In `is_done`, the only lifetime in the signature is that of `&self`, which is elided. The `Formatter<'_>` of the previous section is in there too.

When the `impl` needs a name

The `Iterator` implementation is different. Its associated type `Item` is a `&str` borrowed from the text, not from the iterator, and `rest` returns what is left of the text, also as a borrow of the text. Both need to say "the lifetime of the text", and that requires a name:

```
// The associated type and `rest` mention the lifetime: it needs a name.
impl<'src> Iterator for Words<'src> {
    type Item = &'src str;

    fn next(&mut self) -> Option<Self::Item> {
        let trimmed = self.rest.trim_start();
        let end = trimmed.find(' ').unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        (!word.is_empty()).then_some(word)
    }
}

impl<'src> Words<'src> {
    /// What is left of the text: a borrow of the text, not of `self`.
    fn rest(&self) -> &'src str {
        self.rest
    }
}
```

Since `rest` returns a `&'src str` rather than a borrow of `self`, its result remains usable after the `Words` value is gone:

```
fn main() {
    let text = String::from("parse the first two words only");
    let mut words = Words { rest: &text };
    let first = words.next();
    let second = words.next();
    let rest = words.rest(); // borrows `text`, so `words` can be dropped
    drop(words);
    println!("{first:?} {second:?} then {rest:?}");
}
```

```
    let done = Words { rest: "  " };
    println!("{done} is done: {}", done.is_done());
}
```

```
Some("parse") Some("the") then " first two words only"
Words("") is done: true
```

What happens if you keep the `'_` in the header and try to refer to the lifetime anyway? The program `code/compile_fail/ch06_impl_header_name.rs` has a `Parser` like the one of Chapter 2:

```
impl Parser<'_> {
    pub fn rest(&self) -> &'src str {
        &self.input[self.pos..]
    }
}
```

```
error[E0261]: use of undeclared lifetime name `'src`
  --> compile_fail/ch06_impl_header_name.rs:12:28
   |
12 |     pub fn rest(&self) -> &'src str {
   |                            ^^^^ undeclared lifetime
   |
help: consider introducing lifetime `'src` here
   |
12 |     pub fn rest<'src>(&self) -> &'src str {
   |                ++++++
help: consider introducing lifetime `'src` here
   |
11 | impl<'src> Parser<'_> {
   |     ++++++
```

E0261, "use of undeclared lifetime name", says that `'src` is a name nobody declared: the struct's definition declares a `'src`, but that name is local to the struct definition, and the `impl` declared its own lifetime anonymously. Both suggestions make the error go away, and both are traps. Each introduces a *new* lifetime called `'src`, on the method or on the `impl`, that has nothing to do with the parser's lifetime. Following the second suggestion literally gives `code/compile_fail/ch06_impl_unrelated.rs`:

```
impl<'src> Parser<'_> {
    pub fn rest(&self) -> &'src str {
        &self.input[self.pos..]
    }
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch06_impl_unrelated.rs:13:9
   |
11 | impl<'src> Parser<'_> {
   |      ---- lifetime `'src` defined here
12 |     pub fn rest(&self) -> &'src str {
   |                 ----- has type `&Parser<'1>`
13 |         &self.input[self.pos..]
   |         ^^^^^^^^^^^^^^^^^^^^^^^ method was supposed to return data with lifetime `'src`
but it is returning data with lifetime `'1`
```

Now the signature is valid and the body is wrong. The label "has type `&Parser<'1>`" shows that the compiler calls the anonymous lifetime of the header `'1`, and the message says that the method promised data with lifetime `'src` while returning data with lifetime `'1`. Nothing relates the two. (The first suggestion fails in the same way: a `'src` declared on the method is chosen by each caller of `rest`.)

The fix is to give the lifetime its name where it is introduced, in the header: `impl<'src> Parser<'src>`, as `Words` does above.

Associated types follow the same rule. An associated type is not a function signature, so there is no elision for it, and `'_` is rejected outright (`code/compile_fail/ch06_assoc_type.rs`):

```
impl Iterator for Words<'_> {
    type Item = &'_ str;

    fn next(&mut self) -> Option<Self::Item> {
        let trimmed = self.rest.trim_start();
        let end = trimmed.find(' ').unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        (!word.is_empty()).then_some(word)
    }
}
```

```
error[E0637]: `'_` cannot be used here
  --> compile_fail/ch06_assoc_type.rs:11:18
   |
11 |     type Item = &'_ str;
   |                  ^^ `'_` is a reserved lifetime name
```

E0637, "`'_` cannot be used here", with the label "`'_` is a reserved lifetime name": the compiler will not invent a lifetime for `Item`. The rule of thumb for `impl` headers: write `'_` when the `impl` holds for every lifetime and nothing inside refers to it; declare a name as soon as a method signature, an associated type or a `where` clause must mention it.

6.4 Job 3: letting trait objects borrow

A trait object type always carries a lifetime bound. `dyn Fn(&str) -> String + 'a` means "a value of some type that implements `Fn(&str) -> String` and contains no borrow shorter than `'a`". The bound is needed because the concrete type is erased: the compiler can no longer look inside the value to see what it borrows, so the type must say how long the value may be used. When you do not write a bound, the compiler uses a default, and inside a `Box` the default is `'static`. Chapter 7 met this default in a registry of callbacks; Chapter 8 gives all the rules.

The default is the wrong contract for a function that builds a closure from a borrowed argument (`code/compile_fail/ch06_dyn_static.rs`):

```
pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
    Box::new(move |line| format!("{prefix}{line}"))
}
```

```
error: lifetime may not live long enough
 --> compile_fail/ch06_dyn_static.rs:7:5
  |
6 | pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String> {
  |                              - let's call the lifetime of this reference `'1`
7 |     Box::new(move |line| format!("{prefix}{line}"))
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value requires that
`'1` must outlive `'static`
  |
help: to declare that the trait object captures data from argument `prefix`, you can add an
```

```
explicit `'_` lifetime bound
  |
6 | pub fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
  |                                                                  ++++
```

Read the labels from the top. The compiler calls the lifetime of `prefix` `'1`. The closure captures `prefix` (a `&'1 str`), so the boxed closure contains a borrow with lifetime `'1`. "Returning this value requires that `'1` must outlive `'static`" is the hidden default at work: the return type is really `Box<dyn Fn(&str) -> String + 'static>`. The help gives the fix and names the job: "to declare that the trait object captures data from argument `prefix`, you can add an explicit `'_` lifetime bound". In a return type, `+ '_` is the lifetime that elision picks, here the lifetime of `prefix`. The working version is in `code/examples/ch06_dyn_borrow.rs`:

```
// The closure borrows `prefix`, and `+ '_` says so.
fn make_prefixer(prefix: &str) -> Box<dyn Fn(&str) -> String + '_> {
    Box::new(move |line| format!("{prefix}{line}"))
}
```

`+ '_` in a parameter

In a parameter, `+ '_` follows rule 1 instead: it is a fresh lifetime parameter, not "the same lifetime as something else". A function that runs a list of tasks can accept tasks that borrow anything at all, for the duration of the call:

```
// In a parameter, `+ '_` is a fresh lifetime: tasks that borrow anything at all
// are accepted, for the duration of the call.
fn run_all(tasks: Vec<Box<dyn FnMut() + '_>>) {
    for mut task in tasks {
        task();
    }
}
```

Without `+ '_`, the parameter would be `Vec<Box<dyn FnMut() + 'static>>`, and a task that borrows a local variable would be rejected, just as Chapter 7's `register` rejected a closure that borrows `prefix`. With it, `main` can pass tasks that borrow two local variables mutably:

```
fn main() {
    let prefix = String::from(">> ");
    let quote = make_prefixer(&prefix);
    println!("{}", quote("hello"));

    let mut log = Vec::new();
    let mut runs = 0;
    let tasks: Vec<Box<dyn FnMut() + '_>> =
        vec![Box::new(|| log.push("indexed")), Box::new(|| runs += 1)];
    run_all(tasks);
    println!("{log:?} after {runs} counted run(s)");

    println!("{}", report(&Square(1.5)));
}
```

The `'_` in the `let` of `main` is in a function body, where it means "infer this lifetime" (see below).

`impl dyn Trait + '_`

The default also applies to the type in an `impl` header. An inherent `impl` for a trait object type adds methods that every trait object of that trait can use,

but `impl dyn Shape` means `impl dyn Shape + 'static`, and that is rarely what you want (`code/compile_fail/ch06_impl_dyn_static.rs`):

```
impl dyn Shape {
    pub fn describe(&self) -> String {
        format!("a shape of area {:.2}", self.area())
    }
}

pub fn report(shape: &dyn Shape) -> String {
    shape.describe()
}
```

```
error[E0521]: borrowed data escapes outside of function
  --> compile_fail/ch06_impl_dyn_static.rs:17:5
   |
16 | pub fn report(shape: &dyn Shape) -> String {
   |               -----  - let's call the lifetime of this reference `'1`
   |               |
   |               `shape` is a reference that is only valid in the function body
17 |     shape.describe()
   |     ^^^^^^^^^^^^^^^^
   |     |
   |     `shape` escapes the function body here
   |     argument requires that `'1` must outlive `'static`
   |
note: the used `impl` has a `'static` requirement
  --> compile_fail/ch06_impl_dyn_static.rs:10:10
   |
10 | impl dyn Shape {
   |          ^^^^^ this has an implicit `'static` lifetime requirement
11 |     pub fn describe(&self) -> String {
   |            -------- calling this method introduces the `impl`'s `'static` requirement
help: consider relaxing the implicit `'static` requirement
   |
10 | impl dyn Shape + '_ {
   |                ++++
```

E0521 is "borrowed data escapes outside of function", which sounds more dramatic than it is: nothing is stored anywhere. In `report`, the parameter `&dyn Shape` means `&'1 (dyn Shape + '1)` (Chapter 8 explains why), so the object behind it may borrow data that lives only for `'1`. Calling `describe` requires a `dyn Shape + 'static`, so it requires `'1` to outlive `'static`, and in the compiler's words, the borrow "escapes" into a context that demands more. The note names the culprit: "the used `impl` has a `'static` requirement", "this has an implicit `'static` lifetime requirement". The help relaxes it with `'_`:

```
trait Shape {
    fn area(&self) -> f64;
}

// `impl dyn Shape + '_` is `impl<'a> dyn Shape + 'a`: every trait object.
impl dyn Shape + '_ {
    fn describe(&self) -> String {
        format!("a shape of area {:.2}", self.area())
    }
}

fn report(shape: &dyn Shape) -> String {
    shape.describe()
}
```

In an `impl` header, `'_` is a fresh parameter of the `impl`, so `impl dyn Shape + '_` is `impl<'a> dyn Shape + 'a`: the methods are available on trait objects of every lifetime. The output of the example:

```
>> hello
["indexed"] after 1 counted run(s)
a shape of area 2.25
```

Return-position `impl Trait` used to need `+ '_` for the same reason, and code written for edition 2021 is full of `-> impl Iterator<Item = char> + '_`. In edition 2024 it is rarely needed, because the rules for what an `impl Trait` captures changed; Chapter 8 explains them, together with the `use<..>` syntax that replaced most uses of `+ '_` on `impl Trait`.

6.5 `'_` inside function bodies

Inside a function body, nothing is elided: every lifetime that is not written is *inferred* by the borrow checker from how the values are used (Chapter 1). A type annotation on a `let`, in a turbofish or in a cast can therefore leave all its lifetimes out, and `'_` there means "infer this one". Its use is the same as in signatures, to make a hidden lifetime parameter visible:

```rust
fn record(log: &Mutex<Vec<String>>, line: &str) -> usize {
    // In a body, `'_` stands for "infer this lifetime": the guard borrows `log`.
    let mut guard: MutexGuard<'_, Vec<String>> = log.lock().unwrap();
    guard.push(trimmed(line).to_string());
    guard.len()
}
```

The guard borrows `log`, and `MutexGuard<'_, Vec<String>>` says so. As the second error of `ch06_hidden_param.rs` showed, `elided_lifetimes_in_paths` asks for the `'_` here too. For plain references, `&str` and `&'_ str` mean the same in a body as they do in a signature, so there is no reason to write `'_` after an `&`.

6.6 What `'_` cannot do

It does not resolve ambiguity

When elision has no answer, `'_` has none either. Here is a function with two reference parameters (`code/compile_fail/ch06_ambiguous.rs`):

```rust
// The body only ever returns a piece of `text`, but the signature is read
// without the body: two inputs, so elision has nothing to choose.
pub fn first_word(text: &str, separators: &str) -> &'_ str {
    text.split(|c| separators.contains(c)).next().unwrap_or("")
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch06_ambiguous.rs:8:53
  |
8 | pub fn first_word(text: &str, separators: &str) -> &'_ str {
  |                         ----              ----      ^^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
```

```
say whether it is borrowed from `text` or `separators`
help: consider introducing a named lifetime parameter
  |
8 - pub fn first_word(text: &str, separators: &str) -> &'_ str {
8 + pub fn first_word<'a>(text: &'a str, separators: &'a str) -> &'a str {
  |
```

E0106 is "missing lifetime specifier". The `'_` in the return type asks elision for a lifetime, and with two input lifetimes and no `self`, rule 2 does not apply. The help explains the problem in plain words: "the signature does not say whether it is borrowed from `text` or `separators`". Writing `&str` instead of `&'_ str` gives the same error, because the two spellings mean the same.

It does not look at the body

The body of `first_word` returns only pieces of `text`, so a human can see which input the result borrows from. The compiler does not look. Signatures are contracts, checked on their own: the body is checked against the signature, and callers are checked against the signature and nothing else. If the signature depended on the body, a change inside the function could break its callers, and the compiler could not check callers without reading every function they call.

So the fix must go in the signature, and the help's suggestion deserves a second look: it gives all three references the lifetime `'a`, which compiles but promises too much. Callers would have to keep `separators` alive for as long as they use the result, although the result never borrows it. The honest contract names only the lifetime that the result shares with an input:

```rust
// 2. Two positions that must be the same lifetime: a name. The result borrows
//    `text` only, so `separators` keeps an elided lifetime of its own.
fn first_word<'t>(text: &'t str, separators: &str) -> &'t str {
    text.split(|c| separators.contains(c)).next().unwrap_or("")
}
```

It cannot go in fields, bounds and `where` clauses

`'_` exists to stand for a lifetime that some rule can fill in. Where no rule applies, it is an error:

- In a struct field, there is nothing to elide from: a struct must declare its lifetime parameters, and a field such as `text: &'_ str` is E0106 (see `code/compile_fail/ch03_struct_field_placeholders.rs` in Chapter 3).
- In an associated type of an `impl`, it is E0637, as shown above.
- Inside an `impl Trait` in argument position, such as `words: impl Iterator<Item = &'_ str>`, it is E0658, "anonymous lifetimes in `impl Trait` are unstable". Chapter 8 shows the error and the fix.
- In a bound or a `where` clause, it is E0637 again (`code/compile_fail/ch06_where_clause.rs`):

```rust
pub fn longest_line<I>(lines: I) -> usize
where
    I: Iterator<Item = &'_ str>,
{
    lines.map(str::len).max().unwrap_or(0)
}
```

```
error[E0637]: `'_` cannot be used here
 --> compile_fail/ch06_where_clause.rs:8:25
```

```
  |
8 |     I: Iterator<Item = &'_ str>,
  |                         ^^ `'_` is a reserved lifetime name
```

The fix is a named lifetime parameter: declare `'a` next to `I` and write `I: Iterator<Item = &'a str>`. The iterator yields references of one lifetime, picked by the caller.

6.7 Checklist: `'a`, `'_`, or nothing?

The rules above reduce to a short checklist:

1. **A lifetime that appears once and is never referred to again:** write nothing for a reference, and `'_` for a type path.
2. **Two positions that must be the same lifetime** (two inputs, or an input and the output when elision cannot tell which): a name, as in `first_word` above.
3. **A result that borrows from `self`:** elide it. Rule 3 ties it to `&self`.
4. **A result that borrows from the data a struct borrows**, rather than from `self`: name the struct's lifetime parameter in the `impl` header and use it.
5. **A boxed trait object that borrows an argument:** `+ '_` when elision picks the right input, `+ 'a` otherwise. In an `impl` for a trait object type, `impl dyn Trait + '_`.
6. **In a function body:** nothing, or `'_` where a type path would otherwise hide its lifetime.

The example `code/examples/ch06_checklist.rs` has one function for each of the first five cases (the second is `first_word`, shown above):

```rust
// 1. A lifetime that appears once and is never referred to again: write nothing
//    for a reference, and `'_` for a type path.
fn words(text: &str) -> SplitWhitespace<'_> {
    text.split_whitespace()
}

// 3. The result borrows from `self`: elide (rule 3 of elision).
struct Config {
    name: String,
}

impl Config {
    fn name(&self) -> &str {
        &self.name
    }
}

// 4. The result borrows from the data the struct borrows, not from `self`:
//    name the struct's parameter in the `impl` header.
struct Cursor<'src> {
    rest: &'src str,
}

impl<'src> Cursor<'src> {
    fn take(&mut self, n: usize) -> &'src str {
        let (head, tail) = self.rest.split_at(n.min(self.rest.len()));
        self.rest = tail;
        head
    }
}

// 5. A boxed trait object that borrows an argument: `+ '_`.
```

```
fn matcher(needle: &str) -> Box<dyn Fn(&str) -> bool + '_> {
    Box::new(move |line| line.contains(needle))
}
```

The example's `main` checks the contracts: `key` outlives `separators`, and the two results of `take` are used together, which works only because they borrow the text and not the cursor:

```
fn main() {
    let text = String::from("key=value; other=thing");
    let key;
    {
        let separators = String::from("=;");
        key = first_word(&text, &separators);
    } // `separators` is gone; `key` borrows only `text`
    println!("{key} / {} words", words(&text).count());

    let config = Config {
        name: String::from("parser"),
    };
    println!("{}", config.name());

    let mut cursor = Cursor { rest: &text };
    let (a, b) = (cursor.take(3), cursor.take(1)); // both borrow `text`
    println!("{a:?} {b:?}");

    let has_value = matcher("value");
    println!("{}", has_value(&text));
}
```

```
key / 2 words
parser
"key" "="
true
```

6.8 Common confusions

"**`'_` means 'any lifetime'" or "`'_` means 'infer it'.**" In a signature, `'_` means "elide": a fresh lifetime in a parameter, the lifetime chosen by the elision rules in the output, a fresh parameter in an `impl` header. Only in a function body does it mean "infer".

"**Writing `'_` twice makes the two lifetimes equal.**" Never. Every `'_` is a different lifetime, as `remember` showed. Equality needs a name.

"**`'_` is a short lifetime.**" It is whatever the rules make it. If `make_prefixer` is called with a string literal, the `'_` of its result is `'static`.

"**`Box<dyn Trait + '_>` in a parameter is the same as `Box<dyn Trait>`.**" No: without `+ '_` the parameter accepts only trait objects that borrow nothing shorter than `'static`; with it, any borrow is accepted for the duration of the call.

"**`impl Display for Words<'_>` is a different `impl` from `impl<'a> Display for Words<'a>`.**" They are the same `impl`, written two ways. Writing both is an error, "conflicting implementations" (E0119).

"**The lints are about style.**" They are about style in that the meaning of the program does not change. But a hidden lifetime is a borrow that the signature does not show, and showing it is the purpose of a signature.

6.9 Summary

- '_ is a placeholder, not a name: each occurrence stands for its own lifetime, filled in by the rules of elision for its position. To make two positions share a lifetime, use a name.
- In parameters it is a fresh lifetime; in return types and in + '_ or use<'_> on a return type, it is the lifetime elision picks (E0106 when there is none); in an impl header, a fresh unnameable parameter of the impl; in a body, a lifetime to infer.
- Job 1: make hidden lifetime parameters visible, SplitWhitespace<'_>, Formatter<'_>, MutexGuard<'_, T>. mismatched_lifetime_syntaxes (warn-by-default since 1.89) flags a lifetime written in two different ways between inputs and output; elided_lifetimes_in_paths (allow-by-default, in rust_2018_idioms) flags every hidden one.
- Job 2: in impl headers, '_ replaces a lifetime that nothing inside refers to. A method that returns data with the struct's lifetime, or an associated type that mentions it, needs a named parameter on the impl; the compiler's suggestion to declare a new 'src elsewhere creates an unrelated lifetime.
- Job 3: trait objects default to + 'static in a Box; + '_ lets them borrow. In a parameter it accepts any borrow; impl dyn Trait + '_ makes an inherent impl apply to trait objects of every lifetime.
- '_ never resolves an ambiguous signature and never consults the body. It is not allowed in fields, bounds, where clauses, the associated types of an impl, or argument-position impl Trait.

6.10 Exercises

1. Which of these signatures does mismatched_lifetime_syntaxes flag, and which does elided_lifetimes_in_paths flag? Write each with the hidden lifetimes visible. fn lines_of(s: &str) -> std::str::Lines, fn peek(it: &mut Peekable<Chars>) -> Option<char>, fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result. *Hint:* the first lint needs a lifetime that connects an input to the output; only one of the three has one.
2. The function fn peek_ref(it: &mut Peekable<Chars>) -> Option<&char> is rejected with E0106, although it has a single reference parameter. Explain the error, then write the signature with every lifetime visible and decide which one the result should borrow from. *Hint:* write Chars<'_> and count the lifetimes in the parameter; rule 2 needs exactly one.
3. Expand fn tag(word: &'_ str, out: &mut Vec<&'_ str>) -> Box<dyn Fn() + '_> into named lifetimes. Does it compile? If not, give a signature that does and that lets out be used after the closure is gone. *Hint:* there are three elided input lifetimes. The compiler's suggestion gives all of them one name, &'a mut Vec<&'a str>; Chapter 3 explains why that locks out.
4. Remove the + '_ from impl dyn Shape + '_ in ch06_dyn_borrow.rs. Which calls still compile: describe on a Box<dyn Shape>, on a &dyn Shape parameter, on a &Square coerced in main? Predict, then check. *Hint:* ask, for each receiver, what the object's lifetime bound is; a Box<dyn Shape> written in a signature has the default 'static.

5. In `ch06_impl_headers.rs`, change `rest` to `fn rest(&self) -> &str`, keeping `impl<'src>`. What does the function now promise, and which line of `main` stops compiling? *Hint:* rule 3 ties the result to `&self`; look at the `drop(words)`.